МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Лабораторная работа № 3

по дисциплине «Математические основы баз данных»

по теме: «Многотабличные запросы и аналитические представления»

Выполнил студент группы:
БВТ2505
Пондяков Игорь Дмитриевич
Проверил:
Быватов Семён Юрьевич

Москва 2026

Цель:

Освоить создание многотабличных SQL-запросов и аналитических представлений в СУБД PostgreSQL, научившись соединять данные из нормализованных таблиц, выполнять агрегацию с фильтрацией групп, использовать подзапросы и общие табличные выражения (CTE), а также оформлять типовые аналитические выборки в виде представлений (VIEW).

Задачи:

1. Внутреннее соединение трёх таблиц: Написать запрос с INNER JOIN, объединяющий данные о договорах, абонентах и тарифных планах для получения полной информации об активных подключениях.
2. Левое внешнее соединение: Реализовать запрос с LEFT JOIN для выявления договоров, по которым отсутствуют платежи или заявки в поддержку (поиск «проблемных» или новых клиентов).
3. Агрегация и группировка: Построить запрос с GROUP BY и агрегатными функциями (COUNT, AVG, SUM) для расчёта статистики по категориям услуг или тарифам (количество подключений, средняя выручка).
4. Фильтрация групп (HAVING): Применить секцию HAVING для отбора только тех категорий услуг или тарифов, которые удовлетворяют заданному критерию (например, более 50 активных подключений или средняя цена выше порога).
5. Подзапросы разных видов: Реализовать три отдельных запроса: Скалярный подзапрос (например, поиск договоров с суммой платежей выше средней по базе); Подзапрос с предикатом IN (выборка абонентов, подключивших определённые услуги); Коррелированный подзапрос с EXISTS (поиск сотрудников, исполнявших заявки в конкретный период).
6. Общие табличные выражения (CTE): Использовать WITH для разбиения сложного аналитического запроса на логические шаги; реализовать рекурсивный CTE для обхода иерархии категорий услуг (получение всех подкатегорий заданного уровня).
7. Создание представлений (VIEW): Оформить два наиболее востребованных аналитических отчёта (например, «Сводка по выручке тарифов» и «Статус заявок поддержки») в виде представлений и продемонстрировать обращение к ним.

Выполнение

1. Внутреннее соединение 3 таблиц

Получить сводный реестр активных договоров с полной информацией об абоненте и выбранном тарифном плане. Запрос объединяет данные из трёх нормализованных таблиц (contracts, subscribers, tariffs) по условиям равенства внешних и первичных ключей. Использование INNER JOIN гарантирует, что в результат попадут только те договоры, для которых существуют соответствующие записи во всех связанных таблицах (т.е. исключаются «битые» ссылки или тестовые данные без привязки).

```
SELECT
    c.contract_id,
    s.full_name AS subscriber_name,
    s.phone AS subscriber_phone,
    t.name AS tariff_name,
    t.speed_down || '/' || t.speed_up AS speed_mbps,
    t.monthly_price,
    c.start_date,
    c.status
FROM contracts c
INNER JOIN subscribers s ON c.subscriber_id = s.subscriber_id
INNER JOIN tariffs t ON c.tariff_id = t.tariff_id
WHERE c.status = 'active'
ORDER BY c.start_date DESC;
```

	contract_id integer	subscriber_name character varying (150)	subscriber_phone character varying (20)	tariff_name character varying (100)	speed_mbps text	monthly_price numeric (10,2)	start_date date	status character varying (20)
1	289	Абонент 289	+7900028900	Геймер	300/300	900.00	2026-10-02	active
2	650	Абонент 650	+7900065000	Геймер	300/300	900.00	2026-10-01	active
3	667	Абонент 667	+7900066700	Домашний	100/50	600.00	2026-10-01	active
4	304	Абонент 304	+7900030400	Геймер	300/300	900.00	2026-10-01	active
5	425	Абонент 425	+7900042500	Геймер	300/300	900.00	2026-09-30	active
6	356	Абонент 356	+7900035600	Домашний	100/50	600.00	2026-09-30	active
7	29	Абонент 29	+7900002900	Геймер	300/300	900.00	2026-09-30	active
8	450	Абонент 450	+7900045000	Геймер	300/300	900.00	2026-09-29	active
9	982	Абонент 982	+7900098200	Геймер	300/300	900.00	2026-09-29	active
10	682	Абонент 682	+7900068200	Домашний	100/50	600.00	2026-09-28	active
11	424	Абонент 424	+7900042400	Домашний	100/50	600.00	2026-09-28	active
12	431	Абонент 431	+7900043100	Бизнес	500/500	1500.00	2026-09-28	active
13	756	Абонент 756	+7900075600	Старт	50/20	450.00	2026-09-28	active

2. Запрос с LEFT JOIN

Выявить активные договоры, по которым в таблице платежей (payments) отсутствуют какие-либо записи. Такой запрос позволяет находить новых абонентов, ещё не совершивших первую оплату, или выявлять потенциальные проблемы с биллингом. Использование LEFT JOIN критически важно: внутреннее соединение (INNER JOIN) отфильтровало бы эти договоры, так как для них нет парных строк в правой таблице.

```
SELECT
    c.contract_id,
    s.full_name AS subscriber_name,
    t.name AS tariff_name,
    c.start_date,
    MAX(p.payment_date) AS last_payment_date
    MAX(p.amount) AS last_payment_amount
FROM contracts c
LEFT JOIN payments p ON c.contract_id = p.contract_id
JOIN subscribers s ON c.subscriber_id = s.subscriber_id
JOIN tariffs t ON c.tariff_id = t.tariff_id
WHERE c.status = 'active'
GROUP BY c.contract_id, s.full_name, t.name, c.start_date -- Группируем, чтобы схлопнуть дубли платежей
ORDER BY c.start_date DESC
LIMIT 10
```

	contract_id integer	subscriber_name character varying (150)	tariff_name character varying (100)	start_date date	last_payment_date timestamp with time zone	last_payment_amount numeric
1	289	Абонент 289	Геймер	2026-10-02	2027-01-02 00:00:00+03	900.00
2	667	Абонент 667	Домашний	2026-10-01	2027-01-01 00:00:00+03	600.00
3	650	Абонент 650	Геймер	2026-10-01	2027-01-01 00:00:00+03	900.00
4	304	Абонент 304	Геймер	2026-10-01	2027-01-01 00:00:00+03	900.00
5	425	Абонент 425	Геймер	2026-09-30	2026-12-30 00:00:00+03	900.00
6	356	Абонент 356	Домашний	2026-09-30	2026-12-30 00:00:00+03	600.00
7	29	Абонент 29	Геймер	2026-09-30	2026-12-30 00:00:00+03	900.00
8	450	Абонент 450	Геймер	2026-09-29	2026-12-29 00:00:00+03	900.00
9	982	Абонент 982	Геймер	2026-09-29	2026-12-29 00:00:00+03	900.00
10	431	Абонент 431	Бизнес	2026-09-28	2026-12-28 00:00:00+03	1500.00

3. Агрегация и группировка данных

Получить сводную статистику по категориям дополнительных услуг: количество активных подключений и среднюю ежемесячную выручку на одно подключение. Запрос демонстрирует использование агрегатных

функций (COUNT, AVG) совместно с секцией GROUP BY для вычисления показателей по каждой группе строк. Поскольку одна услуга может быть подключена к множеству договоров (связь M:N через таблицу contract_services), необходимо корректно объединить данные из четырёх таблиц перед группировкой.

```
SELECT
    sc.name AS category_name,
    COUNT(cs.contract_id) AS total_connections,
    ROUND(AVG(s.monthly_price), 2) AS avg_service_revenue
FROM service_categories sc
JOIN services s ON sc.category_id = s.category_id
JOIN contract_services cs ON s.service_id = cs.service_id
JOIN contracts c ON cs.contract_id = c.contract_id
WHERE c.status = 'active'
GROUP BY sc.name
ORDER BY total_connections DESC;
```

	category_name character varying (100)	total_connections bigint	avg_service_revenue numeric
1	Интернет	168	150.00
2	HD Каналы	161	200.00
3	Безопасность	158	100.00

4. Фильтрация групп с помощью HAVING

Определить наиболее востребованные тарифные планы — те, по которым количество активных договоров превышает заданный порог. Запрос демонстрирует применение секции HAVING для фильтрации уже сгруппированных данных, а также показывает ключевое различие между WHERE (фильтр исходных строк до группировки) и HAVING (фильтр агрегированных результатов после группировки).

```
SELECT
    t.name AS tariff_name,
    t.monthly_price,
    COUNT(c.contract_id) AS active_contracts,
    ROUND(AVG(t.monthly_price), 2) AS avg_price_in_group
```

```
FROM tariffs t
JOIN contracts c ON t.tariff_id = c.tariff_id
WHERE c.status = 'active'
GROUP BY t.tariff_id, t.name, t.monthly_price
HAVING COUNT(c.contract_id) > 150
ORDER BY active_contracts DESC;
```

	tariff_name character varying (100)	monthly_price numeric (10,2)	active_contracts bigint	avg_price_in_group numeric
1	Геймер	900.00	257	900.00
2	Домашний	600.00	250	600.00
3	Старт	450.00	155	450.00

5. Подзапросы разных видов (Скалярный, IN, EXISTS)

Продемонстрировать три принципиально разных механизма вложенных запросов для решения аналитических задач интернет-провайдера: поиск значений относительно среднего (скалярный), фильтрация по списку связанных сущностей (IN) и проверка существования связанных записей без их фактического извлечения (EXISTS).

Скалярный подзапрос

```
SELECT name, monthly_price
FROM tariffs
WHERE monthly_price > (SELECT AVG(monthly_price) FROM tariffs);
```

	name character varying (100)	monthly_price numeric (10,2)
1	Геймер	900.00
2	Бизнес	1500.00

Подзапрос с предикатом IN

```
SELECT s.full_name, s.phone
FROM subscribers s
```

```
WHERE s.subscriber_id IN (
    SELECT c.subscriber_id
    FROM contracts c
    JOIN contract_services cs ON c.contract_id = cs.contract_id
    JOIN services serv ON cs.service_id = serv.service_id
    WHERE serv.name = 'Антивирус' AND c.status = 'active'
);
```

	full_name character varying (150)	phone character varying (20)
1	Абонент 617	+7900061700
2	Абонент 455	+7900045500
3	Абонент 781	+7900078100
4	Абонент 552	+7900055200
5	Абонент 70	+7900007000
6	Абонент 52	+7900005200
7	Абонент 867	+7900086700
8	Абонент 861	+7900086100
9	Абонент 539	+7900053900

Коррелированный подзапрос с EXISTS

```
SELECT t.name, t.monthly_price
FROM tariffs t
WHERE EXISTS (
    SELECT 1
    FROM contracts c
    WHERE c.tariff_id = t.tariff_id AND c.status = 'active'
);
```

	name character varying (100)	monthly_price numeric (10,2)
1	Старт	450.00
2	Домашний	600.00
3	Геймер	900.00
4	Бизнес	1500.00

6. Рекурсивный CTE обход дерева категорий

Продемонстрировать два применения конструкции WITH рекурсивный CTE (WITH RECURSIVE) для обхода иерархической структуры категорий услуг с вычислением уровня вложенности и полного пути.

```
WITH RECURSIVE category_tree AS (
    SELECT
        category_id,
        name,
        parent_id,
        0 AS level,
        name::TEXT AS path
    FROM service_categories
    WHERE parent_id IS NULL

    UNION ALL

    SELECT
        sc.category_id,
        sc.name,
        sc.parent_id,
        ct.level + 1,
        ct.path || ' -> ' || sc.name
    FROM service_categories sc
    INNER JOIN category_tree ct ON sc.parent_id = ct.category_id
)
```

```
SELECT
    level,
    path AS category_path,
    name AS category_name
FROM category_tree
ORDER BY path;
```

	level integer	category_path text	category_name character varying (100)
1	0	Безопасность	Безопасность
2	0	Интернет	Интернет
3	1	Интернет -> Оптика	Оптика
4	0	Телевидение	Телевидение
5	1	Телевидение -> HD Кана...	HD Каналы

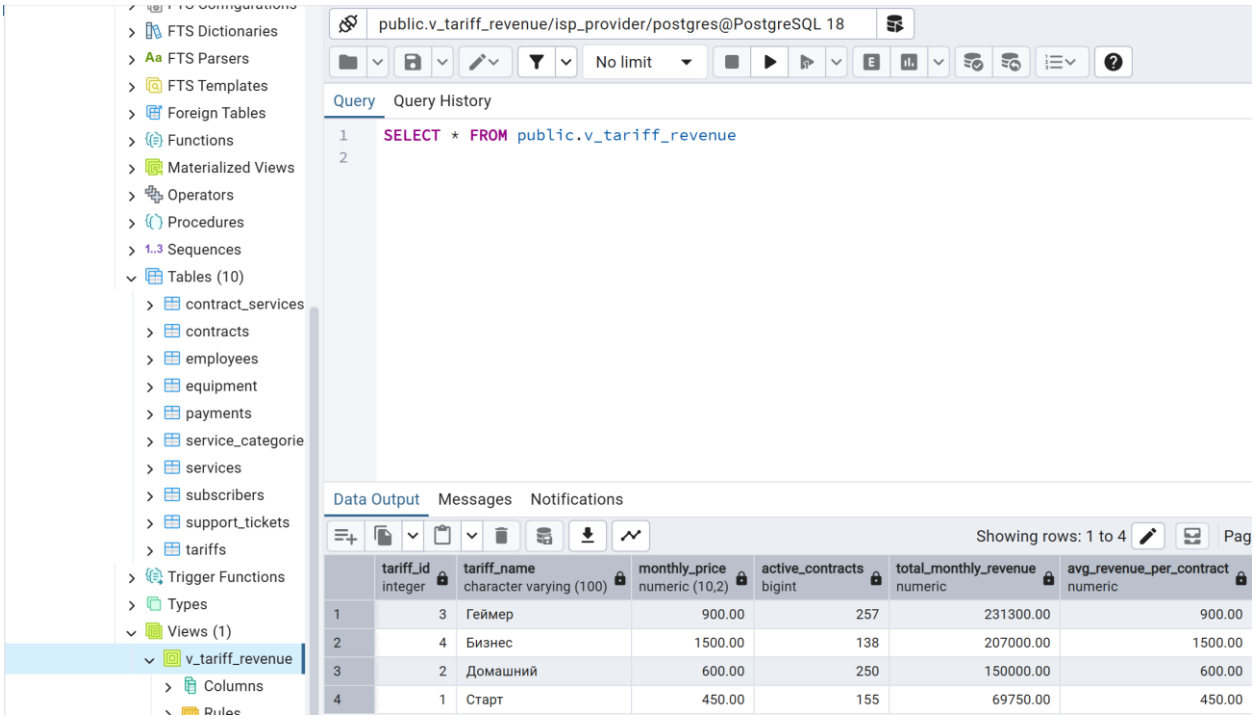
7. Создание представлений (VIEW) для типовых отчётов

Представления (VIEW) сохраняют сложные аналитические запросы в базе данных под именами, позволяя многократно обращаться к ним без повторного написания SQL. Они скрывают сложность соединений и агрегаций, предоставляют готовую «витрину данных» для отчётности и могут использоваться для ограничения доступа к определённым столбцам. В отличие от материализованных представлений, обычный VIEW не хранит данные физически при каждом обращении выполняется лежащий в его основе запрос, что гарантирует актуальность результатов.

Сводка по выручке тарифных планов

```
CREATE OR REPLACE VIEW v_tariff_revenue AS
SELECT
    t.tariff_id,
    t.name AS tariff_name,
    t.monthly_price,
    COUNT(c.contract_id) AS active_contracts,
    ROUND(SUM(t.monthly_price), 2) AS total_monthly_revenue,
    ROUND(AVG(t.monthly_price), 2) AS avg_revenue_per_contract
FROM tariffs t
```

```
JOIN contracts c ON t.tariff_id = c.tariff_id
WHERE c.status = 'active'
GROUP BY t.tariff_id, t.name, t.monthly_price
ORDER BY total_monthly_revenue DESC;
```

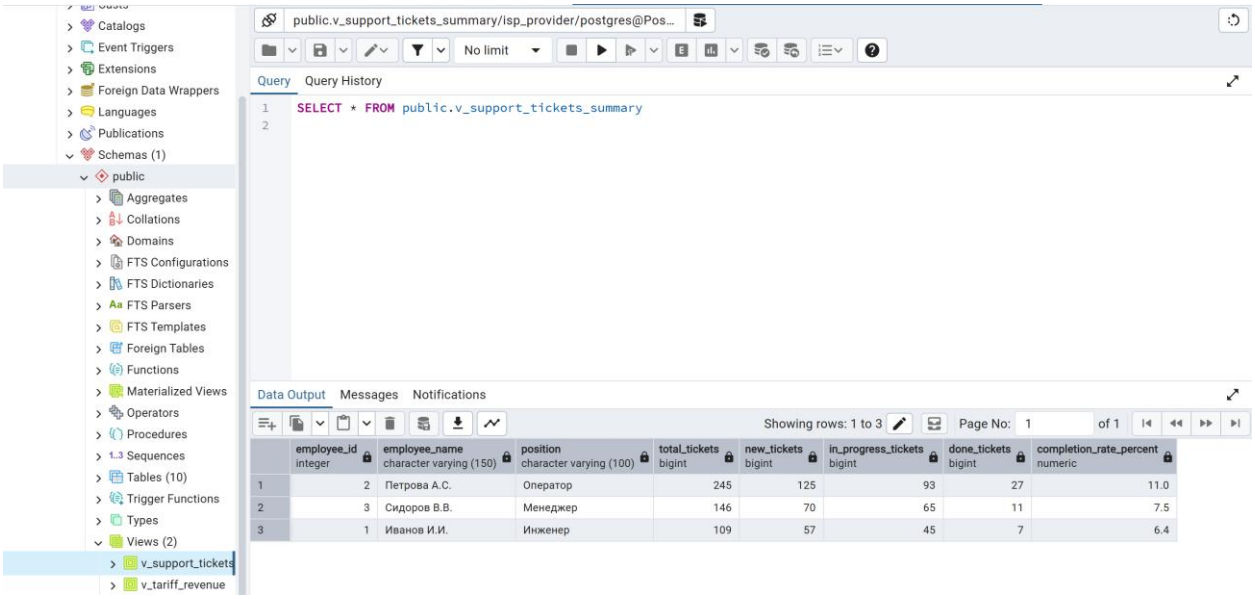


	tariff_id integer	tariff_name character varying (100)	monthly_price numeric (10,2)	active_contracts bigint	total_monthly_revenue numeric	avg_revenue_per_contract numeric
1	3	Геймер	900.00	257	231300.00	900.00
2	4	Бизнес	1500.00	138	207000.00	1500.00
3	2	Домашний	600.00	250	150000.00	600.00
4	1	Старт	450.00	155	69750.00	450.00

Статус заявок технической поддержки

```
CREATE OR REPLACE VIEW v_support_tickets_summary AS
SELECT
    e.employee_id,
    e.full_name AS employee_name,
    e.position,
    COUNT(st.ticket_id) AS total_tickets,
    COUNT(st.ticket_id) FILTER (WHERE st.status = 'new') AS new_tickets,
    COUNT(st.ticket_id) FILTER (WHERE st.status = 'in_progress') AS
in_progress_tickets,
    COUNT(st.ticket_id) FILTER (WHERE st.status = 'done') AS done_tickets,
    ROUND(
        100.0 * COUNT(st.ticket_id) FILTER (WHERE st.status = 'done')
        / NULLIF(COUNT(st.ticket_id), 0),
        1
    ) AS completion_rate_percent
FROM employees e
```

```
LEFT JOIN support_tickets st ON e.employee_id = st.employee_id
GROUP BY e.employee_id, e.full_name, e.position
ORDER BY total_tickets DESC;
```



Вывод

В ходе выполнения лабораторной работы №3 были освоены продвинутые возможности языка SQL для работы с нормализованными реляционными данными в СУБД PostgreSQL. База данных интернет-провайдера, спроектированная в первой работе и наполненная во второй, была использована для решения комплексных аналитических задач.